Guía del proyecto *tareaApis*

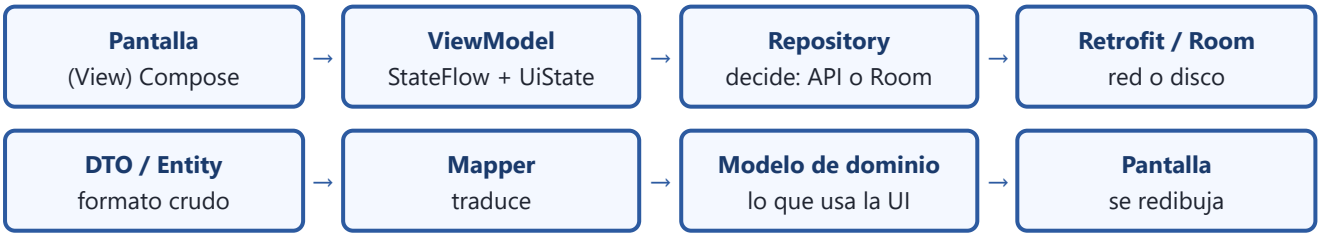
Cómo se aplican **APIs (Retrofit)**, **MVVM** y **Room** en tu código, y qué hay en cada carpeta.

0. Resumen en 30 segundos

La app es un catálogo con 4 pestañas: **Pokémon**, **Rick y Morty**, **Marvel** y **Favoritos**. Cada pestaña muestra una cuadrícula con buscador; al tocar un elemento se abre su detalle. Los Pokémon se pueden marcar como favoritos (con una nota) y se guardan en el teléfono con Room.

Tecnología	Para qué sirve	Dónde vive en tu código
API (Retrofit + OkHttp + Gson)	Pedir datos a internet (JSON) y convertirlos en objetos Kotlin.	`data/api`, `data/dto`
Room	Guardar datos en una base de datos SQLite local (caché y favoritos), que funciona sin internet.	`data/local`
MVVM	Organizar el código en capas para que la pantalla no sepa de dónde vienen los datos.	Model = `data/` + `domain/` · ViewModel = `ui/viewmodel` · View = `ui/screens` y `ui/components`
Jetpack Compose	Dibujar la interfaz con funciones `@Composable` (no hay XML de layouts).	`ui/`, `MainActivity.kt`
Corrutinas y Flow	Trabajo asíncrono (`suspend`) y datos que cambian en el tiempo (`Flow`, `StateFlow`).	Repositorios, DAOs y ViewModels
Coil	Descargar y mostrar imágenes desde una URL (`AsyncImage`).	`ui/components`, `PokeApplication.kt`

El recorrido completo de un dato



Regla de oro: **cada capa solo habla con la siguiente**. La pantalla nunca llama a Retrofit ni a Room directamente.

1. APIs: cómo se aplican en tu código

Una API REST es una URL que responde con JSON. Con **Retrofit** describes esas URLs en una *interfaz* y Retrofit construye el código de red por ti. Son 4 piezas:

1.1 La interfaz con los endpoints (`data/api/PokeApi.kt`)

```
interface PokeApi {
    @GET("pokemon")                       // GET https://pokeapi.co/api/v2/pokemon
    suspend fun getPokemonList(
        @Query("limit") limit: Int,       // ...?limit=1025
        @Query("offset") offset: Int = 0
    ): PokemonListDto

    @GET("pokemon/{id}")                  // {id} se reemplaza por el parámetro
    suspend fun getPokemon(@Path("id") id: Int): PokemonDto

    @GET("pokemon-species/{id}")
    suspend fun getSpecies(@Path("id") id: Int): PokemonSpeciesDto
}
```

- `@GET("...")` : el método HTTP y la ruta (se suma a la URL base).
- `@Path` reemplaza un pedazo de la ruta; `@Query` agrega `?clave=valor` .
- `suspend` : la llamada no bloquea la pantalla; se ejecuta dentro de una corrutina.
- El tipo de retorno (`PokemonDto`) es la clase a la que se convierte el JSON.

Tienes tres interfaces: `PokeApi` , `RickAndMortyApi` y `MarvelApi` (esta usa el repositorio público de Superhero API).

1.2 El cliente único (`data/api/ApiClient.kt`)

```
object ApiClient {
    private val httpClient by lazy {
        OkHttpClient.Builder()
            .connectTimeout(15, TimeUnit.SECONDS)
            .readTimeout(30, TimeUnit.SECONDS)
            .addInterceptor(HttpLoggingInterceptor()...)   // imprime las peticiones en Logcat
            .build()
    }
    private inline fun <reified T> create(baseUrl: String): T =
        Retrofit.Builder()
            .baseUrl(baseUrl)
            .client(httpClient)
            .addConverterFactory(GsonConverterFactory.create())  // JSON -> objetos
            .build().create(T::class.java)

    val pokeApi: PokeApi by lazy { create(POKE_API_URL) }
    ...
}
```

Es un `object` (singleton): hay un solo `OkHttpClient` compartido y solo cambia la URL base por API. `by lazy` significa que se crea la primera vez que se usa. El permiso `INTERNET` está en el `AndroidManifest.xml` ; sin él nada funciona.

1.3 Los DTO (`data/dto/`)

DTO = *Data Transfer Object*: una clase que es un **espejo exacto del JSON**. Gson usa los nombres de las propiedades para llenarlos.

```
data class PokemonDto(
    val id: Int,
    val name: String,
    @SerializedName("base_experience") val baseExperience: Int?,  // el JSON usa snake_case
    val sprites: SpritesDto,
    val types: List<TypeSlotDto>, ...
)
```

- `@SerializedName` conecta el nombre del JSON con el nombre Kotlin.
- Un tipo con `?` (nullable) indica que ese campo puede venir vacío en el JSON.

1.4 El Mapper (`data/mapper/`)

El DTO es feo e incómodo (altura en decímetros, nombres en inglés tipo `special-attack`, textos en 40 idiomas). El **mapper** lo convierte al **modelo de dominio** limpio que usa la pantalla:

```
fun PokemonDto.toDomain(species: PokemonSpeciesDto): Pokemon = Pokemon(
    name = name.prettify(),
    heightMeters = height / 10.0,                 // decímetros -> metros
    description = species.flavorTextEntries.preferSpanish { it.language.name }?.flavorText,
    stats = stats.map { Stat(STAT_NAMES[it.stat.name] ?: ..., it.baseStat) },
    ...
)
```

Así, si la API cambia su formato, solo se toca el mapper y no las pantallas.

1.5 Manejo de errores (`data/repository/SafeApiCall.kt`)

Toda llamada de red puede fallar. `safeApiCall` envuelve la llamada y devuelve un `Result` (éxito o fallo) con un mensaje comprensible:

```
suspend fun <T> safeApiCall(notFoundMessage: String = "...", block: suspend () -> T): Result<T> =
    try { Result.success(block()) }
    catch (e: CancellationException) { throw e }          // nunca tragarse la cancelación
    catch (e: HttpException) { ... 404 / 429 / 5xx ... }    // el servidor respondió con error
    catch (e: IOException)   { Result.failure(Exception("Sin conexión a internet")) }
```

Idea clave: en vez de lanzar excepciones hacia arriba, el repositorio devuelve `Result<T>`. El ViewModel decide si mostrar los datos o el error.

1.6 Peticiones en paralelo (`PokemonRepository.getPokemon`)

```
val pokemon = coroutineScope {
    val pokemon = async { api.getPokemon(id) }     // las dos peticiones salen a la vez
    val species = async { api.getSpecies(id) }
    pokemon.await().toDomain(species.await())      // se juntan al terminar ambas
}
```

2. Room: cómo se aplica en tu código

Room es una capa cómoda sobre SQLite. Tú escribes **clases y anotaciones**; Room genera el SQL y el código al compilar (por eso está `kapt("androidx.room:room-compiler")` en `app/build.gradle.kts`). Hay 3 piezas obligatorias:

Pieza	Qué es	Tu archivo
Entity	Una clase = una tabla. Cada propiedad = una columna.	`FavoritoEntity`, `PokemonCacheEntity`, `PokemonDetalleEntity`
DAO	Interfaz con las consultas (*Data Access Object*).	`FavoritoDao`, `PokemonCacheDao`
Database	Clase abstracta que une entidades y DAOs.	`PokemonDatabase`

2.1 Entity (`data/local/FavoritoEntity.kt`)

```
@Entity(tableName = "favoritos")
data class FavoritoEntity(
    @PrimaryKey val id: Int,                // el id del Pokémon ya es único: sirve de clave
    val nombre: String,
    val imagenUrl: String,
    val agregadoEn: Long = System.currentTimeMillis(),   // para ordenar por fecha
    val nota: String? = null                             // opcional (añadida en la versión 3)
)
```

Tienes tres tablas: `favoritos` (decisiones del usuario), `pokemon_cache` (la lista de 1025 Pokémon) y `pokemon_detalle` (el detalle guardado como texto JSON). Están **separadas a propósito**: la caché se puede borrar y reconstruir; los favoritos no deben perderse nunca.

2.2 DAO (`data/local/FavoritoDao.kt`)

```
@Dao
interface FavoritoDao {
    @Query("SELECT * FROM favoritos ORDER BY agregadoEn DESC")
    fun observarFavoritos(): Flow<List<FavoritoEntity>>          // Flow: se reemite solo al
cambiar la tabla

    @Query("SELECT id FROM favoritos")
    fun observarIdsFavoritos(): Flow<List<Int>>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertar(favorito: FavoritoEntity)              // suspend: escritura fuera del
hilo principal

    @Query("DELETE FROM favoritos WHERE id = :id")
    suspend fun eliminarPorId(id: Int)

    @Query("UPDATE favoritos SET nota = :nota WHERE id = :id")
    suspend fun actualizarNota(id: Int, nota: String?)
}
```

- **Consultas que devuelven** `Flow`: son *reactivas*. Si alguien inserta o borra un favorito, Room vuelve a ejecutar la consulta y la pantalla se actualiza sola. Por eso el corazón cambia al instante en todas las pestañas.
- **Funciones** `suspend` (insertar, borrar, actualizar): Room las ejecuta en un hilo de fondo.
- `:id` / `:nota` en el SQL se enlazan con los parámetros de la función.

2.3 DAO con transacción (`PokemonCacheDao.kt`)

```
@Transaction
open suspend fun reemplazarTodos(pokemones: List<PokemonCacheEntity>) {
    borrarTodo()
    insertarTodos(pokemones)
}
```

`@Transaction` hace que DELETE + INSERT sean **todo o nada**: si algo falla a mitad, la caché vieja queda intacta. (Por eso este DAO es `abstract class` y no `interface`: tiene una función con cuerpo.)

2.4 Database + Singleton + Migraciones (`PokemonDatabase.kt`)

```
@Database(
    entities = [FavoritoEntity::class, PokemonCacheEntity::class, PokemonDetalleEntity::class],
    version = 4, exportSchema = false
)
abstract class PokemonDatabase : RoomDatabase() {
    abstract fun favoritoDao(): FavoritoDao
    abstract fun pokemonCacheDao(): PokemonCacheDao
    companion object {
        fun obtener(context: Context): PokemonDatabase =      // Singleton con doble verificación
            instancia ?: synchronized(this) { instancia ?: crear(context, "pokedex.db").also {
instancia = it } }
    }
}
```

- **Singleton**: abrir la base de datos es caro; se usa una sola instancia para toda la app (`@Volatile` + `synchronized` evitan crear dos).
- **Migraciones** (`MIGRACION_1_2` , `2_3` , `3_4`): cuando cambias el esquema (nueva tabla o columna) subes `version` y escribes el SQL que transforma la base vieja en la nueva **sin borrar los datos del usuario**. Tu historial: v1 favoritos → v2 + caché de lista → v3 + columna `nota` → v4 + tabla de detalle.

2.5 Cómo usa Room el Repositorio: «fuente única de verdad»

```
suspend fun getPokemonList() = safeApiCall {
    if (cacheDao.contar() == 0) downloadList()   // solo la primera vez baja de internet
    readCachedList()                              // SIEMPRE se lee desde Room
}

suspend fun getPokemon(id: Int) = safeApiCall {
    val cached = cacheDao.obtenerDetalle(id)      // ¿ya lo tengo guardado?
    if (cached != null) gson.fromJson(cached, Pokemon::class.java)
    else { ...pide a la API...; cacheDao.guardarDetalle(PokemonDetalleEntity(id,
gson.toJson(pokemon))); pokemon }
}
```

Patrón «cache-first»: primero Room, la API solo rellena. Por eso la Pokédex abre sin internet después de la primera vez. El botón **Actualizar** llama a `refreshList()`, que descarga primero y reemplaza la caché dentro de una transacción.

2.6 Prueba de la base de datos

`androidTest/.../PokemonDatabaseTest.kt` crea una base *en memoria* (`Room.inMemoryDatabaseBuilder`) para probar favoritos, caché y migraciones sin tocar la base real. Estos tests necesitan emulador/dispositivo.

3. MVVM: cómo se aplica en tu código

MVVM separa responsabilidades en tres capas:

Capa	Responsabilidad	En tu proyecto
Model	Datos y reglas: de dónde vienen (API/Room), cómo se transforman.	`data/` (api, dto, local, mapper, repository) y `domain/model`
View	Dibujar y recibir toques. **No tiene lógica de negocio.**	`ui/screens`, `ui/components`, `MainActivity`
View**M**odel	Guarda el estado de la pantalla y responde a las acciones. Sobrevive a rotaciones.	`ui/viewmodel`

3.1 UiState: los 3 estados de toda pantalla (`ui/viewmodel/UiState.kt`)

```
sealed interface UiState<out T> {
    data object Loading : UiState<Nothing>
    data class Success<T>(val data: T) : UiState<T>
    data class Error(val message: String) : UiState<Nothing>
}
fun <T> Result<T>.toUiState(): UiState<T> = fold(onSuccess = { UiState.Success(it) }, onFailure =
{ UiState.Error(it.message ?: "...") })
```

La pantalla solo tiene que preguntar «¿estoy cargando, tengo datos o hubo error?». Aquí se ve el puente: `Result` (capa de datos) → `UiState` (capa de UI).

3.2 El ViewModel genérico (`CatalogViewModel.kt`)

Las 3 pantallas de catálogo hacen lo mismo (cargar lista, buscar, abrir detalle), así que hay **un solo ViewModel genérico** y cada API solo le entrega funciones:

```
open class CatalogViewModel<Item, Detail>(
    private val loadItems: suspend () -> Result<List<Item>>,    // cómo cargar la lista
    private val loadDetail: suspend (id: Int) -> Result<Detail>,  // cómo cargar un detalle
    private val matches: (item: Item, query: String) -> Boolean,  // cómo filtrar con el buscador
    private val refreshItems: (suspend () -> Result<List<Item>>)? = null
) : ViewModel()

class MarvelViewModel(repository: MarvelRepository = MarvelRepository())
    : CatalogViewModel<Hero, Hero>(
        loadItems = repository::getHeroes,
        loadDetail = repository::getHero,
        matches = { hero, query -> hero.name.contains(query, ignoreCase = true) ... })
```

3.3 StateFlow: el estado observable

```
private val allItems = MutableStateFlow<UiState<List<Item>>>(UiState.Loading)   // privado y
modificable
private val _query   = MutableStateFlow("")
val query: StateFlow<String> = _query.asStateFlow()                            // público y de
solo lectura

// Lista visible = lista completa + texto del buscador, recalculada automáticamente
val items: StateFlow<UiState<List<Item>>> = combine(allItems, _query) { state, query ->
    if (state is UiState.Success && query.isNotBlank()) UiState.Success(state.data.filter {
matches(it, query.trim()) })
    else state
}.stateIn(viewModelScope, SharingStarted.Eagerly, UiState.Loading)
```

- **Patrón `_privado` / `público`**: solo el ViewModel puede modificar el estado; la View solo lo lee.
- `combine`: mezcla dos flujos y recalcula cuando cualquiera cambia.
- `viewModelScope.launch { }`: lanza corrutinas que se cancelan solas cuando el ViewModel muere.
- **Flujo de datos unidireccional**: el estado baja hacia la View; los eventos (`onQueryChange`, `openDetail`, `refresh`) suben al ViewModel.

3.4 ViewModel que usa Room (`FavoritesViewModel.kt`)

```
class FavoritesViewModel(application: Application) : AndroidViewModel(application) {
    private val repository =
FavoritesRepository(PokemonDatabase.obtener(application).favoritoDao())
    private val sortByName = MutableStateFlow(false)

    val uiState: StateFlow<FavoritesUiState> = sortByName
        .flatMapLatest { byName ->                       // al cambiar el orden, se cambia la
consulta Room
            repository.observeFavorites(byName).map { FavoritesUiState(favorites = it, loading =
false, ...) } }
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), FavoritesUiState())

    fun toggle(pokemon: PokemonSummary) { ... if (isFavorite) repository.remove(id) else
repository.add(pokemon) }
}
```

Aquí no hay red. Room emite un `Flow`, el ViewModel lo convierte en `StateFlow` y la pantalla lo pinta. Al tocar el corazón se escribe en Room → Room reemite → el corazón cambia solo (no hay que actualizar nada a mano). `AndroidViewModel` se usa porque necesita el `Application` para abrir la base de datos.

3.5 Factory: crear un ViewModel con dependencias (`PokemonViewModel.kt`)

```
val Factory: ViewModelProvider.Factory = viewModelFactory {
    initializer {
        val app = this[APPLICATION_KEY] as Application
        PokemonViewModel(PokemonRepository(PokemonDatabase.obtener(app).pokemonCacheDao()))
    }
}
```

Android crea los ViewModel por sí mismo y solo sabe usar constructores vacíos. Como `PokemonViewModel` necesita un repositorio (que necesita Room, que necesita el contexto), se le entrega una *fábrica* que dice

cómo construirlo. Rick y Morty y Marvel tienen valores por defecto en el constructor, por eso no la necesitan.

3.6 La View (`ui/screens` + `ui/components`)

```
@Composable
fun PokemonScreen(
    viewModel: PokemonViewModel = viewModel(factory = PokemonViewModel.Factory),
    favoritesViewModel: FavoritesViewModel = viewModel()
) {
    val favoriteIds by favoritesViewModel.favoriteIds.collectAsStateWithLifecycle()   //
StateFlow -> State de Compose
    CatalogScreen(viewModel = viewModel, ..., itemContent = { pokemon, onClick -> CatalogCard(...
isFavorite = pokemon.id in favoriteIds, onFavoriteClick = { favoritesViewModel.toggle(pokemon) })
})
}
```

- `collectAsStateWithLifecycle()`: convierte un `StateFlow` en estado de Compose. Cuando el valor cambia, Compose **redibuja** lo que lo usa. Además deja de escuchar cuando la app está en segundo plano.
- `CatalogScreen` (`ui/components/Catalog.kt`) es la View genérica: recibe el ViewModel y dos lambdas (`itemContent`, `detailContent`) que dibujan la tarjeta y el detalle de cada API. `StateContent` (en `Common.kt`) decide qué dibujar según `Loading / Error (con botón Reintentar) / Success`.
- `CatalogCard` «no sabe de Room»: solo recibe `isFavorite` y avisa con `onFavoriteClick`. Esto es **state hoisting**: los componentes no guardan estado, lo reciben y avisan.
- `MainActivity` contiene `App()`: un `Scaffold` con barra superior, barra de navegación inferior y un `when(selected)` que muestra la pantalla de la pestaña activa. Comparte una misma instancia de `FavoritesViewModel` entre pestañas.

3.7 Ejemplo completo: tocar un corazón

1. El usuario toca ♥ en `CatalogCard` → llama `onFavoriteClick`.
2. `PokemonScreen` ejecuta `favoritesViewModel.toggle(pokemon)`.
3. `FavoritesViewModel` lanza una corrutina y llama a `FavoritesRepository.add(...)`.
4. El repositorio convierte `PokemonSummary` → `FavoritoEntity` con el mapper y llama a `dao.insertar(...)`.
5. Room escribe en SQLite y **reemite** los `Flow` de la tabla `favoritos`.
6. `favoriteIds` y `uiState` cambian → Compose redibuja el corazón (y el contador «♥ N» de la barra, y la pestaña Favoritos).

3.8 Ejemplo completo: abrir la Pokédex

1. `PokemonViewModel` se crea → su `init` llama `reloadItems()` → `allItems = Loading` (la pantalla muestra el indicador).
2. `PokemonRepository.getPokemonList()` (dentro de `safeApiCall`): si Room está vacío, Retrofit descarga 1025 Pokémon, el mapper los convierte y se guardan en `pokemon_cache`.
3. Se lee la lista desde Room y se devuelve como `Result.success`.
4. `toUiState()` → `UiState.Success(lista)` → la pantalla dibuja la cuadrícula con imágenes (Coil).

5. Si no hay internet y la caché está vacía: `Result.failure("Sin conexión a internet")` → `UiState.Error` → botón **Reintentar**.

4. Qué hay en cada carpeta

Ruta base del código: `app/src/main/java/com/example/tareaapis/`

```
tareaApis/
├── app/
│   ├── build.gradle.kts            ← dependencias y configuración del módulo
│   └── src/
│       ├── main/
│       │   ├── AndroidManifest.xml
│       │   ├── java/com/example/tareaapis/
│       │   │   ├── MainActivity.kt, PokeApplication.kt
│       │   │   ├── data/      (api · dto · local · mapper · repository)
│       │   │   ├── domain/    (model)
│       │   │   └── ui/        (components · screens · theme · viewmodel)
│       │   └── res/           ← recursos (colores, textos, íconos)
│       ├── test/              ← pruebas unitarias (en tu PC)
│       └── androidTest/       ← pruebas instrumentadas (en emulador)
├── gradle/libs.versions.toml   ← catálogo de versiones
├── build.gradle.kts, settings.gradle.kts
└── app/build/ , .idea/ , .gradle/   ← generadas, no se editan
```

4.1 Archivos en la raíz del paquete

Archivo	Qué hace
`MainActivity.kt`	Punto de entrada de la app (única Activity). `setContent` carga el tema y `App()`, que arma las 4 pestañas con `Scaffold` + `NavigationBar`.
`PokeApplication.kt`	Clase `Application` (se crea antes que cualquier pantalla; declarada en el Manifest). Configura el cargador de imágenes de Coil con `respectCacheHeaders(false)` para reutilizar imágenes guardadas aunque «venzan».

4.2 Carpeta `data/` — la capa Model (de dónde vienen los datos)

Subcarpeta	Contenido	Se guarda aquí
`data/api`	`PokeApi`, `RickAndMortyApi`, `MarvelApi` (interfaces Retrofit) y `ApiClient` (configuración única de red).	Los **endpoints** y la configuración de red.
`data/dto`	`PokemonDto`, `RickMortyDto`, `MarvelDto`.	Clases que son **copia fiel del JSON** de cada API.
`data/local`	`PokemonDatabase`, DAOs (`FavoritoDao`, `PokemonCacheDao`) y Entities (`FavoritoEntity`, `PokemonCacheEntity`, `PokemonDetalleEntity`).	Todo lo de **Room**: tablas, consultas y migraciones.

`data/mapper`	`PokemonMapper`, `RickMortyMapper`, `MarvelMapper`, `MapperUtils`.	Funciones de extensión que **convierten** DTO/Entity ↔ modelo de dominio. `MapperUtils` trae ayudas: `prettify()`, `idFromUrl()`, `cleanOrNull()`.
`data/repository`	`PokemonRepository`, `RickMortyRepository`, `MarvelRepository`, `FavoritesRepository`, `SafeApiCall`.	La **lógica de acceso a datos**: decide si leer de internet, de Room o de memoria, y devuelve `Result`.

4.3 Carpeta `domain/model/`

`Models.kt` contiene los modelos «limpios» que usa la interfaz: `PokemonSummary`, `Pokemon`, `FavoritePokemon`, `RickMortyCharacter`, `Hero`, `Stat`, y enums como `CharacterStatus`. No dependen de Retrofit, ni de Room, ni de Compose: son Kotlin puro. Es lo único que la UI conoce.

4.4 Carpeta `ui/` — View + ViewModel

Subcarpeta	Contenido	Se guarda aquí
`ui/viewmodel`	`CatalogViewModel` (genérico), `PokemonViewModel`, `RickMortyViewModel`, `MarvelViewModel`, `FavoritesViewModel`, `UiState`.	El **estado de cada pantalla** y la lógica de reacción a eventos.
`ui/screens`	`PokemonScreen`, `RickMortyScreen`, `MarvelScreen`, `FavoritesScreen`.	Una pantalla por pestaña: arma la tarjeta y el detalle específicos de cada API usando los componentes comunes.
`ui/components`	`Catalog.kt` (`CatalogScreen`, `CatalogCard`), `Detail.kt` (cabecera, secciones, barras de estadísticas, píldoras), `Common.kt` (`StateContent`, `LoadingBox`, `EmptyBox`, `SearchField`).	**Piezas visuales reutilizables** sin lógica de datos.
`ui/theme`	`Color.kt`, `Type.kt`, `Theme.kt`.	Colores, tipografía y el tema Material 3 (`TareaApisTheme`), incluido modo claro/oscuro.

4.5 Carpeta `res/` (recursos)

Subcarpeta	Qué guarda
`drawable/`, `mipmap-*/`	Íconos de la app (`ic_launcher`) en distintas densidades de pantalla.
`values/`	`strings.xml` (textos, p. ej. `app_name`), `colors.xml`, `themes.xml` (tema base de la Activity).
`xml/`	`backup_rules.xml` y `data_extraction_rules.xml`: qué datos se respaldan en la nube / al cambiar de teléfono.

No hay carpeta `layout/` porque la interfaz se escribe con Compose en Kotlin, no en XML.

4.6 Pruebas y archivos de configuración

Ruta	Qué es
`app/src/test`	Pruebas unitarias que corren en la JVM de tu PC (rápidas). Hoy solo trae el ejemplo.
`app/src/androidTest`	Pruebas instrumentadas en emulador/teléfono. Incluye `PokemonDatabaseTest` para Room.
`app/build.gradle.kts`	SDK mínimo (24) y objetivo (35), plugins (Compose, kapt) y **dependencias**: Retrofit, OkHttp, Coil, Room, Compose, Lifecycle.
`gradle/libs.versions.toml`	Catálogo central de versiones de librerías (`retrofit = "2.11.0"` , etc.).
`settings.gradle.kts` / `build.gradle.kts` (raíz)	Define qué módulos tiene el proyecto y plugins globales.
`AndroidManifest.xml`	Declara el permiso `INTERNET` , la clase `PokeApplication` y `MainActivity` como pantalla de arranque (`LAUNCHER`).
`app/build/` , `.gradle/` , `.idea/`	Generadas por Gradle y Android Studio. **No se editan ni se suben a git.**

5. Observaciones sobre tu código

- **Archivos sin uso:** `AppDatabase.kt` , `FavoriteDao.kt` y `FavoriteEntity.kt` (en `data/local`) no los usa nada del proyecto. Parecen un intento anterior; la base real es `PokemonDatabase` con `FavoritoDao` / `FavoritoEntity` . Son seguros de borrar y evitan confusión (hay dos «favoritos» casi iguales).
- **Idioma mezclado:** en `data/local` los nombres están en español (`observarFavoritos` , `nombre`) y en el resto en inglés (`observeFavorites`). Funciona, pero conviene elegir uno.
- **Caché en memoria:** Rick y Morty y Marvel guardan la lista en una variable del repositorio (se pierde al cerrar la app). Solo Pokémon usa Room. Es una buena práctica de estudio extender Room a las otras dos.
- **Instancias de repositorio:** `RickMortyViewModel` y `MarvelViewModel` crean su repositorio por defecto; en proyectos grandes se usa inyección de dependencias (Hilt/Koin).

6. Glosario rápido

Término	Significado
DTO	Clase que refleja el JSON de la API.
Entity	Clase que refleja una tabla de Room.

DAO	Interfaz con las consultas a una base de datos.
Repository	Clase que decide de dónde salen los datos (API, Room, memoria).
Mapper	Función que convierte entre DTO/Entity y modelo de dominio.
Flow / StateFlow	Flujo de valores que cambian; `StateFlow` siempre guarda el último valor y la UI lo observa.
suspend	Función que puede pausarse sin bloquear el hilo (corrutinas).
viewModelScope	Ámbito de corrutinas atado a la vida del ViewModel.
Composable	Función que describe un trozo de interfaz; se vuelve a ejecutar cuando cambia el estado que lee.
Result<T>	Éxito con valor o fallo con excepción, sin lanzar errores hacia arriba.
Singleton	Una sola instancia compartida (`object` o patrón `obtener()`).